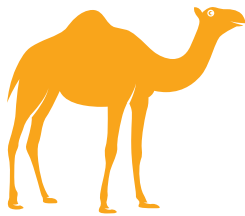


Implementing Parallel Processing with Apache Camel

Apache Camel offers various components and enterprise integration patterns (EIPs) to achieve concurrency. This article explains the various options available and the best practices to be followed to achieve high scalability when using these EIPs.



Apache Camel provides a number of EIPs (listed below) that allow a main route to divide processing across multiple sub-routes:

- Multicast
- Split
- recipientList
- wireTap

Some of these EIPs provide parallel processing support out-of-the-box, helping to achieve high scalability. Camel ships with default config settings for these EIPs, which can be tuned further to suit one's requirements to get better performance. This article discusses some such useful settings that provide better performance when tuned.

To better illustrate these options let us consider the code snippet given below that uses Camel's Multicast EIP.

We will explore the multiple tuning options available for this EIP and, using sample code snippets, explain how they can be tuned:

```
From ("mainRoute")
    .multicast()
    .aggregationStrategy (new
ResponseAggregator())
    .parallelProcessing()
    .to ("subRoute1", "subRoute2",
subroute3")
```

A message arriving at route 'mainRoute' is being multicast to three different sub-routes which process it in parallel, and once done with processing, the response is aggregated using the aggregation strategy defined in *ResponseAggregator* class. Now let us look at the various fine-tuning options available to achieve high concurrency and scalability out of the above implementations.

a. Custom thread pool

When using Multicast EIP for parallel processing, Camel uses a default thread pool which has a maximum pool size

of 20, limiting the number of parallel threads that can be spanned to 20:

```
<threadPoolProfile
id="defaultThreadPoolProfile"
defaultProfile="true"

poolSize="10" maxPoolSize="20"
maxQueueSize="1000"

allowCoreThreadTimeOut="false"
rejectedPolicy="CallerRuns"/>
```

With these pool size settings, multicast invocation becomes a major performance bottleneck when processing higher transactions per second (TPS). Also, in cases like the above, where the sub-routes are calls to other components, the pool threads will be majorly in a waiting state and get exhausted quickly. This will result in incoming requests simply waiting for pool threads to free up, leading to an increase in response time and decreased throughput.

It's recommended to use a custom thread pool tuned for the performance needs of each use case rather than using default thread pool settings. All the parallel processing EIPs mentioned above provide a mechanism for passing a custom thread pool. There are two ways to customise a processor's thread pool.

Approach 1: Specify a custom thread pool—explicitly create an *ExecutorService* (thread pool) instance and pass it to the *executorService* option. For example:

```
from ("mainRoute")
    .multicast()
```

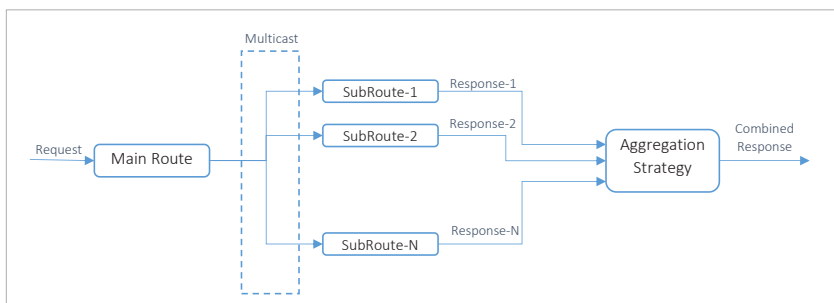


Figure 1: Camel parallel processing